
MILESTONE 1 — EXECUTION DOCUMENT

SupportPilot

Ticket Processing & Classification — Weeks 1–2 Build Specification

DOCUMENT TYPE	Milestone execution spec — form fields, classification methods, UI pages, backend modules, day plan
PROJECT CODE	SP-ATR-001 / M1
DURATION	10 working days (Weeks 1–2)
EXIT CRITERIA	≥ 90% of tickets correctly categorised • Severity prediction accuracy ≥ 85%
SCOPE NOTE	Knowledge retrieval and resolution generation are Milestone 2. M1 delivers intake, classification, severity, priority and SLA only.
AUDIENCE	Frontend devs, Python devs, ML dev, QA, Tech Lead

Table of Contents

1. What Milestone 1 Must Deliver

Scope boundary and exit criteria

2. Category Taxonomy & Priority Model

Define this before writing any code

3. Complete Form Field Specification

Every field, type, validation, consumer

4. Classification Method Specification

Every method: input, output, logic, location

5. Reference Data Required

What must exist before the model can be evaluated

6. Frontend Build List

Pages, components, folder placement, build order

7. Python Backend Build List

Modules, responsibilities, endpoints

8. Classification & Severity Models

Baseline vs ML, features, evaluation protocol

9. Day-by-Day Plan

10 working days, task allocation

10. Definition of Done Checklist

Signed off before Milestone 2 starts

1. What Milestone 1 Must Deliver

1.1 The Scope Boundary

The temptation in this milestone is to start on RAG, because "classification is easy and retrieval is the interesting part". Resist it. Classification quality determines everything downstream — a ticket routed to the wrong category retrieves the wrong knowledge and generates the wrong resolution. Get this right first.

IN Milestone 1	NOT in Milestone 1 (Milestone 2 or later)
Ticket submission form, fully validated	Knowledge base articles, ingestion, embeddings
Email and API ingestion endpoints	Vector search, RAG pipeline, reranking
Text preprocessing and PII masking	Resolution generation, citations
Category and sub-category classification	Multi-agent orchestration (Diagnosis, Retrieval, Resolution)
Severity prediction with business overrides	Jira integration, email automation
Deterministic priority mapping (P1-P4)	Escalation workflow, routing rules engine
SLA due-date computation with business calendars	Analytics dashboard, rollups, exports
Ticket state machine and status history	Confidence-based auto-resolve gate
Agent-facing ticket list, queue and detail views	Agent workbench with run traces
Manual classification override, captured as a label	Retraining pipeline

The one thing that must ship even though it looks like Milestone 2 work

Manual classification override. Every time an agent corrects the AI's category or severity, that correction is a labelled training example. If you do not capture overrides from day one of production use, you arrive at Milestone 4 with no feedback data and no way to improve the model. It is a small feature with disproportionate long-term value — build it in M1.

1.2 Exit Criteria and How Each Is Measured

Criterion	Measurement	Evidence required at review
Ticket classification coverage	Labelled hold-out set of 500 historical tickets, never seen in training	Evaluation report: overall accuracy $\geq$ 90%, plus per-class confusion matrix
Severity prediction accuracy	Same hold-out set, severity label	Accuracy $\geq$ 85%, plus a breakdown of which direction errors go
Intake operational	Portal, email and API paths each create a ticket	Live demo: submit via all three channels, all appear in the agent queue classified

On the confusion matrix: overall accuracy hides the failure that matters. A model that scores 91% while systematically confusing Network with VPN is worse than one scoring 89% with errors spread evenly, because the Network/VPN confusion will route tickets to the wrong team every time. Report per-class precision and recall, not just the headline number.

2. Category Taxonomy & Priority Model

This is a decision, not code. Make it in a one-hour session on day 1 with someone who actually works the support desk, write it down, and freeze it for the milestone. Changing the taxonomy after training invalidates every label you have collected.

2.1 Category Taxonomy

Category	Sub-categories	Default routing team
NETWORK	Connectivity, WiFi, LAN, DNS, Bandwidth	Network Team
VPN	Connection failure, Certificate, Client install, Timeout	Network Team
ACCESS	Password reset, Account lockout, Permissions, MFA, Onboarding	Identity Team
SOFTWARE	Installation, Licensing, Crash, Update, Compatibility	Desktop Support
HARDWARE	Laptop, Desktop, Peripheral, Docking station, Mobile device	Desktop Support
PRINTER	Not printing, Driver, Queue stuck, Quality, Scan	Desktop Support
EMAIL	Mailbox, Distribution list, Spam, Calendar, Storage quota	Messaging Team
SECURITY	Phishing report, Malware, Suspicious activity, Data request	Security Team
APPLICATION	ERP, CRM, Internal tool, Integration failure, Performance	Application Support
UNCLASSIFIED	—	General Queue

Two taxonomy rules that prevent most later pain
First,
UNCLASSIFIED must exist as a real class
, not an error state. The model will encounter tickets that genuinely do not fit, and forcing a guess is worse than admitting uncertainty — those tickets route to the general queue where a human sorts them, which is exactly the correct outcome. Second,
keep VPN separate from NETWORK
even though it is technically a subset. VPN is high-volume, highly templated, and the single best auto-resolution candidate in Milestone 2. Merging it into NETWORK buries it.

2.2 Severity Definitions

Severity	Definition	Typical example
CRITICAL	Business-wide outage or security incident. Work has stopped for many people.	Site-wide network down, active malware spread, ERP unavailable
HIGH	A team or a critical individual cannot work. No workaround exists.	Department printer down before a deadline, VP laptop failure, VPN down for a remote team
MEDIUM	Work is degraded but continuing. A workaround exists.	Slow application, one user's software crash, peripheral fault
LOW	Request or minor inconvenience. No work is blocked.	Software install request, access request, how-to question

2.3 Priority Mapping Matrix

Priority is **not** a model output. It is a deterministic lookup from severity and business impact. Keeping it deterministic means SLA assignment is auditable and a manager can always explain why a ticket is P1.

Severity ↓ / Impact →	Single user	Team (2-20)	Department (21-100)	Org-wide (100+)
CRITICAL	P2	P1	P1	P1
HIGH	P3	P2	P1	P1
MEDIUM	P4	P3	P2	P2
LOW	P4	P4	P3	P3

2.4 SLA Policy (seed values)

Priority	First response	Resolution	Calendar
P1	15 minutes	4 hours	24×7 — clock never pauses

P2	1 hour	8 business hours	Business hours
P3	4 business hours	2 business days	Business hours
P4	1 business day	5 business days	Business hours

3. Complete Form Field Specification

This is the contract between frontend and backend. Every field has a Zod schema entry, a serializer field, and a database column.

3.1 Section 1 — The Issue

Field	Control	Required	Validation	Consumed by
subject	Text	Yes	5-150 characters. Reject "help", "urgent", "issue" alone.	Classifier (primary signal)
description	Textarea, 6 rows	Yes	Min 20 characters. Placeholder prompts for error message, what they tried, when it started.	Classifier, Severity, RAG (M2)
category_hint	Dropdown	No	Taxonomy list + "Not sure" default. A <i>hint</i> , not the answer.	Classifier (feature)
affected_system	Text with autocomplete	No	Suggests from a known application list.	Classifier, routing
started_when	Dropdown	No	Just now / Today / This week / Longer / Recurring	Severity (feature)

Do not let the user set priority directly

The obvious design is a "How urgent is this?" dropdown feeding straight into priority. Do not do it. Every ticket becomes Critical within two weeks — not because users are dishonest, but because their own blocked work genuinely feels urgent to them. Capture self-reported urgency as an *optional feature the model consumes*, weighted low, and let the model plus deterministic rules decide the actual severity. If you must show something, show the resulting priority read-only with an explanation.

3.2 Section 2 — Impact

Field	Control	Required	Validation	Consumed by
affected_scope	Radio group	Yes	Just me / My team / My department / Whole organisation	Priority matrix (direct input)
is_work_blocked	Radio: Yes / Partially / No	Yes	—	Severity (strong feature)
self_reported_urgency	Dropdown	No	Low / Normal / High. Low weight in the model.	Severity (weak feature)
workaround_available	Checkbox	No	—	Severity (feature)

`affected_scope` and `is_work_blocked` are the two questions that do most of the severity work, and both are one tap. Two well-chosen structured fields beat a paragraph of free text for this specific purpose.

3.3 Section 3 — Context

Field	Control	Required	Validation	Consumed by
department	Dropdown, prefilled from profile	Yes	Editable if the user is raising on behalf of another team.	Classifier (feature), routing
location_site	Dropdown, prefilled	No	Office/site list.	Routing, severity (site outage detection)
asset_tag	Text	No	Format check against the asset scheme.	Routing, future CMDB link
attachments	File upload, multi	No	Max 5 files, 10 MB each. Images, PDF, log files, .txt. Virus scanned.	Classifier (filename features)
contact_preference	Radio	No	Email / Phone / Teams. Default email.	Notifications (M3)
best_time_to_contact	Text	No	Max 60 characters.	Assignment context

3.4 Section 4 — Requester (auto for portal, required for email/API)

Field	Control	Required	Validation	Consumed by
requester_email	Read-only from session	Yes	Must resolve to a user record on API ingestion.	Identity, notifications
requester_name	Read-only from session	Yes	—	Display
on_behalf_of	User search, behind a	No	Must resolve to an active user. Agents	Identity, notifications

	toggle		raising for others.	
--	--------	--	---------------------	--

3.5 Field Behaviour Rules

Rule	Implementation
Validate on blur	Zod resolver with mode: 'onBlur'. Never surface six errors at submit.
Live classification preview	Debounce 800 ms after the user stops typing in description, call the classify endpoint and show the predicted category as a soft hint with a confidence bar. Do not block submission on it.
Duplicate detection	On blur of subject, check the requester's open tickets for near-duplicates. Show "You have a similar open ticket — add to it instead?" This deflects real volume at zero cost.
Description prompts	Placeholder text that asks the three questions worth asking: what is the error message, what did you already try, when did it start. Better input produces better classification.
Draft persistence	Debounce localStorage write every 2 s.
Never ask for derivable data	Department, site, email, name all come from the user profile. Priority is computed. Category is predicted.
Submit is fast	Persist and return 201 immediately, then classify asynchronously. The user must never wait on a model.

4. Classification Method Specification

Every method below is a pure function where possible: no HTTP calls inside the calculation, no framework imports. This makes them unit-testable and lets the same logic run in a batch evaluation script.

4.1 Method Index

Method	Module	Returns
strip_email_artifacts	preprocessing.py	Clean body text
mask_pii	preprocessing.py	Text with PII replaced by tokens
normalise_text	preprocessing.py	Model-ready text
extract_features	features.py	Feature vector for the fast path
classify_fast	classifier/predict.py	Category, sub-category, confidence
classify_llm	classifier/llm_path.py	Same shape, from structured LLM output
classify	classifier/dispatch.py	Routed result with the path recorded
predict_severity	severity/predict.py	Severity + confidence
apply_severity_overrides	severity/rules.py	Final severity after business rules
map_priority	priority.py	P1-P4 from the matrix
compute_sla_due	sla.py	First-response and resolution due timestamps
find_duplicates	dedup.py	Similar open tickets for the requester
generate_ticket_number	numbering.py	Human-facing reference
validate_transition	state_machine.py	Allowed or rejected with a reason

4.2 Preprocessing

strip_email_artifacts(raw_text) → text

1. Remove quoted reply blocks: lines starting with ">", blocks after "On <date> <person> wrote:", "-----Original Message-----" 2. Remove signature blocks: after "--", "Regards,", "Thanks,", "Sent from my" 3. Remove disclaimer footers: "This email and any attachments..." 4. Collapse whitespace, strip leading/trailing blank lines 5. Cap at 4000 characters (keep the head, it carries the problem statement)

Email-sourced tickets are typically 80% quoted history and signature. Feeding that to a classifier buries the actual signal. This function is unglamorous and it materially moves accuracy on the email channel.

mask_pii(text) → (masked_text, mask_map)

Pattern	Replaced with	Note
Email addresses	<EMAIL>	Preserve the domain if internal — it is a useful feature
Phone numbers	<PHONE>	E.164 and local formats
IP addresses	<IP>	Keep whether it was private vs public range as a feature
Employee IDs	<EMPID>	Match the organisation's ID scheme
Anything after "password is / pwd:"	<REDACTED>	Users do paste passwords into tickets. Mask before storage, not just before the LLM call.
Card-like number sequences	<CARD>	Luhn check to avoid masking order numbers

The `mask_map` is retained in memory for the request only, so display can unmask for the agent. Never persist the map.

4.3 Feature Extraction

`extract_features(ticket) → vector` Text features: `embedding(subject + " " + description)` ← sentence embedding, 384 or 768 dim OR `tfidf(subject, weight 2.0) + tfidf(description, weight 1.0)` Structured features: department one-hot channel one-hot (PORTAL / EMAIL / API) affected_scope ordinal (1..4) is_work_blocked ordinal (0, 1, 2) self_reported_urgency ordinal (1..3), low weight workaround_available binary started_when ordinal has_attachment binary attachment_ext_flags binary set (.log, .png, .txt) subject_length numeric description_length numeric has_error_code binary (regex: 0x[0-9A-F]+, ERR_\w+, code \d{3,5}) keyword_flags binary set (vpn, printer, password, outlook, wifi, install, license, locked) requester_top_category categorical, from their last 10 tickets hour_of_day, day_of_week cyclical encoding

Subject is weighted higher than description because users write their clearest statement of the problem in the subject line and then ramble. The keyword flags look primitive next to embeddings, but they carry exact tokens ("VPN", error codes) that embeddings smooth

away, and they cost nothing.

4.4 Classification Dispatch

```
classify(ticket) → ClassificationResult clean = normalise_text(mask_pii(strip_email_artifacts(ticket.description))) features =
extract_features(ticket with clean text) fast = classify_fast(features) route to LLM if ANY of: fast.confidence < 0.75 fast.category
has < 30 training examples ticket.channel == EMAIL and description > 1500 chars keyword_flags contains a security term if routed:
slow = classify_llm(ticket, taxonomy) if slow is valid: result = slow, path = "LLM" else: result = fast, path = "FAST_FALLBACK"
else: result = fast, path = "FAST" if result.confidence < 0.40: result.category = "UNCLASSIFIED" return result + { path,
model_version, prompt_version, latency_ms }
```

Why not send every ticket to the LLM

Roughly 70–80% of IT tickets are near-duplicates of previous tickets. A supervised classifier answers those in under 50 ms at effectively zero marginal cost, while an LLM call costs 1–3 seconds and real money per ticket. At 500 tickets a day the difference is material. Route to the LLM only where the classifier is genuinely uncertain — that is where the LLM's reasoning earns its cost. Record which path each ticket took so you can tune the threshold with evidence.

classify_llm — prompt contract

System: You are an IT support ticket classifier. Return ONLY valid JSON matching the schema. Choose from the provided taxonomy only. If no category fits, return "UNCLASSIFIED". Input: taxonomy (category + sub-category list) ticket subject, cleaned description, department (ticket text delimited and labelled as untrusted user data) Output: { category, sub_category, severity, confidence, reasoning } Validation: parse against a Pydantic schema. On parse failure, one repair attempt with the error message appended, then fall back to fast path.

4.5 Severity and Priority

apply_severity_overrides(predicted, ticket, requester) → severity

```
severity = predicted # Rules can only RAISE severity, never lower it. if requester.is_vip: severity = max(severity, HIGH) if
ticket.affected_scope == ORG_WIDE: severity = max(severity, HIGH) if matches(text, OUTAGE_PHRASES): severity = max(severity, HIGH)
if category == SECURITY: severity = max(severity, HIGH) if matches(text, CRITICAL_PHRASES): severity = CRITICAL if
open_similar_tickets_last_hour >= 5: severity = max(severity, HIGH) # incident signal OUTAGE_PHRASES = "site down", "everyone",
"whole team", "cannot work", "production down", "all users" CRITICAL_PHRASES = "data breach", "ransomware", "entire office",
"system-wide outage"
```

Rules raise, never lower. A model that under-predicts severity on a genuine outage is the expensive failure; a model that over-predicts costs an agent five minutes. Make the asymmetry explicit in code. The "5 similar tickets in an hour" rule is worth building — it is how you catch an emerging incident before anyone declares one.

map_priority(severity, affected_scope) → P1..P4

Pure matrix lookup from section 2.3. No model, no heuristics, no exceptions. If someone asks for a special case, it becomes a row in the matrix, not an `if` statement somewhere else.

compute_sla_due(priority, created_at, calendar) → (first_response_due, resolution_due)

```
policy = sla_policies.get(priority) if policy.calendar == "24x7": first_response_due = created_at + policy.first_response_minutes
resolution_due = created_at + policy.resolution_minutes else: first_response_due = add_business_minutes(created_at,
policy.first_response_minutes, calendar) resolution_due = add_business_minutes(created_at, policy.resolution_minutes, calendar)
```

`add_business_minutes` steps forward skipping non-working hours, weekends and holidays. A P3 ticket raised at 4:50 pm on Friday is not due at 8:50 pm Friday. Store a per-site calendar — offices in different countries have different weekends.

4.6 Duplicate Detection

```
find_duplicates(subject, description, requester_id) → List[Ticket] candidates = open tickets where requester_id matches and
created_at > now - 7 days for each candidate: score = 0.6 × cosine(embedding(new), embedding(candidate)) + 0.4 ×
token_overlap(subject_new, subject_candidate) return candidates where score > 0.80, sorted desc, max 3
```

Scoped to the same requester deliberately. Cross-requester duplicate detection is a different feature (incident correlation) with different thresholds — do not conflate them.

5. Reference Data Required

Assign this to a named person on day 1. It is the critical path for the entire milestone — without labelled data, neither exit criterion can be evidenced.

Dataset	Volume needed	Source	Notes
Historical tickets, labelled	≥ 2,500 (2,000 train / 500 hold-out)	Existing ticketing system export	Critical path. Needs subject, description, final category, final severity, department, channel, created date.
Category taxonomy	~10 categories, ~45 sub-categories	Session with the support lead, day 1	Frozen for the milestone once agreed
Users and departments	Full org list	HR export or Active Directory	Includes VIP flag and site assignment
Teams and queues	6-10	Support lead	Maps categories to routing targets
SLA policies	4 rows	Seed from section 2.4, confirm with the lead	Editable later via admin
Business calendars	Per site	HR holiday lists	Working hours plus holidays per country
Application catalogue	50-150	IT asset list	Powers the affected_system autocomplete

Contingency if labelled historical tickets do not arrive
Escalate on day 3, not day 8. The fallback plan: ship the **LLM-only classification path**
for Milestone 1 with no supervised model, since an LLM with a well-written taxonomy prompt reaches roughly 80-88% without any training data. Evidence the exit criteria against a manually labelled set of 200 tickets that the team labels itself in a two-hour session — smaller than ideal but defensible. Then collect agent overrides through M1 and M2 and train the supervised fast path in Milestone 3. State this change openly at the milestone review rather than quietly redefining the criterion.

6. Frontend Build List

6.1 Structural Rule

Pages compose, features own logic, `lib/` owns framework-free helpers. Nothing under `pages/` contains an API call or a business rule. Enforce with an ESLint boundary rule so it survives deadline pressure.

```
src/
├── app/
│   ├── router.tsx                ← route tree + role guards
│   ├── providers/{Query,Auth,Theme,Toast}Provider.tsx
│   └── layouts/{AuthLayout,PortalLayout,ConsoleLayout}.tsx
├── pages/
│   ├── auth/LoginPage.tsx
│   ├── portal/
│   │   ├── NewTicketPage.tsx    ← the submission form
│   │   ├── MyTicketsPage.tsx
│   │   └── TicketTrackPage.tsx
│   ├── dashboard/OverviewPage.tsx ← shell + basic counts only in M1
│   └── tickets/
│       ├── TicketListPage.tsx
│       ├── TicketQueuePage.tsx
│       └── TicketDetailPage.tsx
├── features/
│   ├── auth/
│   │   ├── api/authApi.ts
│   │   ├── hooks/{useLogin,useSession,usePermissions}.ts
│   │   └── components/RoleGuard.tsx
│   ├── tickets/
│   │   ├── api/{ticketApi,ticketKeys}.ts
│   │   ├── hooks/{useTickets,useTicket,useCreateTicket,useTransition}.ts
│   │   └── components/
│   │       ├── TicketForm.tsx
│   │       ├── IssueSection.tsx    ← form section 1
│   │       ├── ImpactSection.tsx  ← form section 2
│   │       ├── ContextSection.tsx ← form section 3
│   │       ├── AttachmentUploader.tsx
│   │       ├── DuplicateWarning.tsx
│   │       ├── TicketTable.tsx
│   │       ├── TicketFilterBar.tsx
│   │       ├── TicketTimeline.tsx
│   │       ├── StatusBadge.tsx
│   │       ├── PriorityChip.tsx
│   │       └── schemas/ticketSchema.ts ← Zod, mirrors the backend contract
│   └── classification/
│       ├── api/classificationApi.ts
│       ├── hooks/useLiveClassification.ts ← debounced preview
│       └── components/
│           ├── ClassificationPanel.tsx
│           ├── ConfidenceMeter.tsx
│           ├── SeverityBadge.tsx
│           ├── ReclassifyDialog.tsx ← captures the override label
│           └── ClassificationPathChip.tsx ← shows FAST vs LLM
├── components/{ui,common,data}/
├── lib/{axios,queryClient,rbac,formatters,constants}.ts
├── store/{authStore,uiStore}.ts
└── config/{routes,navigation,permissions}.ts
```

6.2 Pages to Build in Milestone 1

Page	Contains	Done when
/login	Credential form, token handling, refresh interceptor	Session persists across reload; expired token silently refreshes
/portal/tickets/new	Three-section form, live classification preview, duplicate warning, attachments, draft persistence	All fields in section 3 present with correct conditional behaviour; submit returns in under 1 s
/portal/tickets	Requester's own tickets, status chips, SLA countdown	Scoped correctly — a user cannot see another user's ticket by changing the URL
/portal/tickets/:id	Read-only detail, timeline, classification shown in plain language	No internal fields (confidence, model version, path) leak to the requester view
/tickets	Agent master list: server-side pagination, multi-facet filters, saved views	Filter by status, priority, category, department, assignee, date range all work server-side
/tickets/queue	Work queue ordered by SLA risk; claim action	Ordering reflects time-to-breach, not creation date
/tickets/:id	Two-pane: ticket + comments, and the classification panel with confidence, path, severity and reclassify action	Overriding the classification persists an override record with the previous value
/dashboard	Shell with four counter tiles only	Tiles render; full analytics is M4 and must not be started

6.3 Component Build Order

Order	Component	Why this position
1	ticketSchema (Zod)	Defines the contract. Both teams build against it. Nothing starts before this is agreed.
2	Auth + RoleGuard	Every subsequent page needs a session and a role to render correctly.
3	ui/ primitives + PriorityChip, StatusBadge	Used by every list and detail view. Building them once prevents six divergent badge implementations.
4	Form sections 1-3 + TicketForm	The core intake path. Test with the API mocked.
5	TicketTable + FilterBar	Needs real records to be meaningful, so follows the form.
6	ClassificationPanel + ConfidenceMeter	Needs the classify endpoint live.
7	ReclassifyDialog	Small, but it is the feedback loop. Do not let it slip to M2.
8	DuplicateWarning, live classification preview	Polish. Cut these first if the milestone is at risk.

7. Python Backend Build List

7.1 Core API Modules (Django + DRF)

Module	Owns	Milestone 1 deliverable
apps/accounts/	Identity, RBAC	User model with role, department, site, is_vip. JWT login and refresh. Permission classes for Employee, Agent, Engineer, Lead, Admin. Role-scoped querysets.
apps/masterdata/	Org structure	Department, Team, TeamMembership, Site, BusinessCalendar, ApplicationCatalogue. Import management commands.
apps/tickets/	Ticket lifecycle	Ticket model matching the field spec exactly. State machine with guarded transitions. Comments, attachments via pre-signed URLs, status history. CRUD, filters, cursor pagination. process_ticket Celery task.
apps/classification/	Classification records	TicketClassification model, CategoryTaxonomy, override endpoint that records the previous value as a labelled example.
apps/sla/	SLA policy	SlaPolicy model, compute_sla_due, add_business_minutes, breach-sweep task scaffolding.
apps/audit/	Activity log	Append-only ActivityLog, middleware capturing actor and request ID.
core/	Shared plumbing	Enums as the single source of truth, exception handler producing the uniform error envelope, pagination classes, base model with timestamps.

7.2 AI Service Modules (FastAPI)

Module	Milestone 1 deliverable
app/core/	Config via pydantic-settings, service-to-service auth, structured JSON logging with trace ID
app/preprocessing/	strip_email_artifacts, mask_pii, normalise_text — each with unit tests over real messy samples
app/features/	extract_features, embedding client with batching and retry
app/ml/classifier/	train.py, predict.py, labels.py, baseline.py (keyword rules)
app/ml/severity/	train.py, predict.py, rules.py for the override logic
app/ml/registry.py	Model version tags, load by tag, artifact pull from object storage
app/llm/	base.py LLMProvider protocol plus one implementation, token guard, response cache
app/prompts/	Versioned YAML: classification.v1.yaml. Registry loader.
app/classifier/dispatch.py	The fast/slow routing logic from section 4.4
app/guardrails/	Output schema validation, injection filter on ticket text, PII check before any external call
app/api/v1/routes/	health.py, classify.py
app/evaluation/	Golden set loader, metrics.py (accuracy, macro-F1, confusion), run_eval.py invocable from CI

7.3 Milestone 1 Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/login, /refresh, /logout	Token issue, rotation, revocation
GET	/api/v1/auth/me	Profile, role, department, effective permissions
POST	/api/v1/tickets	Create. Returns 201 immediately, queues classification.
GET	/api/v1/tickets	List with filters, role-scoped queryset, cursor pagination
GET	/api/v1/tickets/{id}	Detail with classification and timeline
PATCH	/api/v1/tickets/{id}	Update mutable fields
POST	/api/v1/tickets/{id}/transition	Guarded status change; 409 on illegal transition
POST	/api/v1/tickets/{id}/comments	Comment with public/internal visibility flag
POST	/api/v1/tickets/{id}/attachments	Request pre-signed upload URL
POST	/api/v1/tickets/check-duplicates	Called on subject blur from the form
POST	/api/v1/tickets/ingest	Service-token endpoint for email poller and external desks
POST	/api/v1/tickets/{id}/classify	Force re-classification
PATCH	/api/v1/classifications/{id}	Manual override — records the previous value as a training label

GET	/api/v1/taxonomy/categories	Category tree for dropdowns
GET	/api/v1/masterdata/applications	affected_system autocomplete source
POST	/v1/classify (AI service)	Internal: category, sub-category, severity, confidence, path
POST	/v1/preview-classify (AI service)	Internal: lightweight fast-path only, for the live form preview

On the live classification preview endpoint

Keep it separate from `/v1/classify` and make it fast-path only. The preview fires on every debounced keystroke pause across every user filling a form — routing that to an LLM would multiply cost by an order of magnitude for a feature that is only a soft hint. The real classification runs once, asynchronously, after submission.

8. Classification & Severity Models

8.1 Three Implementations, One Interface

All three sit behind `predict(features) → result` and are selected by configuration. Build them in this order so something always works.

Implementation	Method	Role
Keyword baseline	Ordered regex rules mapping phrases to categories	Day-3 deliverable. Establishes the pipeline and gives a floor to beat. Reaches roughly 60–70%.
Supervised fast path	LightGBM or logistic regression over embeddings + structured features	The production default. Target 90%+.
LLM slow path	Structured-output prompt with the taxonomy	Low-confidence fallback and cold-start substitute.

8.2 Training Protocol

- 1. Split by time, not randomly.** Train on older tickets, hold out the most recent 500. Random splitting leaks near-duplicate tickets across the boundary and inflates accuracy by several points.
- 2. Handle class imbalance.** Password resets may be 30% of volume and Security 1%. Use class weights, not oversampling — oversampling near-duplicate ticket text overfits badly.
- 3. Two models, not one multi-task model.** Category and severity have different feature importances: category is almost entirely text-driven, severity is driven by `affected_scope` and `is_work_blocked`. Separate models are simpler to debug and to improve independently.
- 4. Calibrate confidence.** Raw classifier probabilities are poorly calibrated. Apply Platt scaling or isotonic regression on a validation slice, otherwise the 0.75 routing threshold is meaningless.
- 5. Evaluate against the baseline first.** Record keyword-baseline accuracy before training. If the supervised model is not clearly better, something is wrong with the features or the labels — investigate rather than shipping it.
- 6. Write the report into the repo.** Model version, feature list, split dates, per-class precision/recall, confusion matrix, calibration curve. This is the milestone evidence.

8.3 What to Inspect in the Confusion Matrix

Pattern	What it means and what to do
NETWORK ↔ VPN confusion	Expected and the most common. Add explicit VPN keyword flags; consider a two-stage classifier that separates these after a coarse pass.
SOFTWARE ↔ APPLICATION confusion	Usually a taxonomy problem, not a model problem. If humans cannot label these consistently, the boundary is wrong — merge or redefine.
Everything → the largest class	Class weighting failed or the model is undertrained. Check the weight configuration first.
High accuracy, low macro-F1	The model is excellent on common classes and useless on rare ones. Accept it for volume classes but route rare-class predictions to the LLM path.
Severity errors skewed low	Dangerous direction. Tighten the override rules and consider shifting the decision threshold to favour higher severity.

9. Day-by-Day Plan

Day	Backend / Python	Frontend	Data / ML
1	Repo, Docker Compose (Postgres, Redis, MinIO, Mailhog), Django skeleton, settings split, Celery wired	Vite + TS project, Tailwind tokens, layouts, router, ui/ primitives	Taxonomy session with the support lead. Request the historical ticket export — escalate immediately if unavailable
2	Accounts app: user model with role/department/site/is_vip, JWT, permission classes. Masterdata models	ticketSchema (Zod) agreed with backend. Login page, session handling, RoleGuard	Load users, departments, teams, sites. Explore the ticket export: label quality, class distribution
3	Tickets model matching the field spec, state machine, POST/GET/PATCH, filters, cursor pagination	Form sections 1 and 2 against a mocked API. PriorityChip, StatusBadge, ConfidenceMeter	Keyword baseline classifier implemented. Baseline accuracy measured and recorded
4	FastAPI skeleton, service auth. preprocessing/ with unit tests over real messy email samples	Form section 3, AttachmentUploader with pre-signed URLs, draft persistence	Clean and split the dataset by time. Build the 500-ticket hold-out and freeze it
5	features/ extraction, embedding client. LLM provider interface plus one implementation	TicketForm assembled end to end. Submit path working against the real API	Feature engineering. Train the first supervised classifier. Compare against baseline
6	classifier/dispatch.py fast/slow routing. /v1/classify. Classification models in Django	TicketTable, FilterBar, TicketListPage with server-side filtering	Severity model training. Confidence calibration. Threshold tuning for the routing rule
7	Severity overrides, priority matrix, SLA computation with business calendars. process_ticket task end to end	TicketDetailPage two-pane layout, ClassificationPanel, ClassificationPathChip	Full evaluation run. Confusion matrix analysis. Fix the top two confusion pairs
8	Ingest endpoint for email/API. Duplicate detection endpoint. Audit middleware	ReclassifyDialog with override capture. TicketQueuePage ordered by SLA risk	Retrain with fixes. Re-evaluate. Lock the shipping model version
9	Contract tests, role-permission matrix tests, seed demo data, fix defects	Portal pages (MyTickets, TicketTrack), dashboard shell, empty and loading states	Write the evaluation report into the repo. Prepare the evidence pack
10	Buffer and defect fixing	Playwright journey, accessibility pass, live classification preview if time permits	Demo rehearsal. Milestone review

The two days that decide the milestone

Day 1 (taxonomy frozen + data requested) and Day 4 (hold-out set built and frozen). If the taxonomy shifts after day 3, every label collected becomes suspect. If the hold-out set is not frozen and isolated, the evaluation number will not be trustworthy and the milestone cannot be signed off honestly. Protect both.

10. Definition of Done Checklist

10.1 Functional

☐	Item
☐	All fields in section 3 exist with correct required/optional behaviour and blur validation
☐	User cannot set priority directly; priority is computed and shown read-only with an explanation
☐	Ticket submission returns 201 in under 1 second; classification runs asynchronously
☐	Tickets arrive via all three channels (portal, email ingest, API) and all are classified
☐	Email-sourced tickets have quoted history and signatures stripped before classification
☐	PII is masked before storage and before any external model call
☐	Severity overrides raise but never lower; unit tested for each rule
☐	Priority follows the matrix exactly; no special-case branches outside it
☐	SLA due dates respect business calendars; a Friday-evening P3 is not due Friday night
☐	UNCLASSIFIED is produced for low-confidence tickets and routed to the general queue
☐	Agent can override category and severity; the override persists with the previous value
☐	Requester views expose no confidence scores, model versions or internal comments
☐	A user cannot access another user's ticket by changing the URL
☐	Illegal state transitions return 409 with a stable error code

10.2 Evidence Required at Review

☐	Item
☐	Evaluation report: category accuracy $\geq$ 90% on the frozen 500-ticket hold-out
☐	Evaluation report: severity accuracy $\geq$ 85%, with the error-direction breakdown
☐	Per-class confusion matrix, with a written note on the top two confusion pairs and what was done
☐	Baseline comparison: keyword accuracy vs shipped model accuracy
☐	Fast/LLM path split percentage and average latency per path
☐	Live demo performed by someone who did not build it: submit $\rightarrow$ classified $\rightarrow$ queue $\rightarrow$ override

10.3 Engineering Quality

☐	Item
☐	Preprocessing, features, priority and SLA modules contain zero framework imports
☐	Unit coverage $\geq$ 80% on preprocessing, severity rules, priority matrix, SLA and state machine
☐	Permission tests parametrised across every endpoint $\times$ every role
☐	Enums defined once in core/enums.py and mirrored in the frontend constants file
☐	OpenAPI contract published and matching the implemented endpoints
☐	Model artifacts versioned in object storage, loaded by tag, version recorded on every classification
☐	Evaluation script runnable from CI with a single command
☐	Classification failure never blocks ticket creation — verified by killing the AI service mid-test

10.4 Carry-Forward Register

Anything incomplete is recorded here with an owner and target milestone rather than silently absorbed. Reviewed at the start of Milestone 2 planning.

Item	Owner	Target	Blocker

